

```
1 ### Cell 1: Imports
2 begin
3     using Pkg
4     Pkg.activate(".")
5     Pkg.instantiate()
6     using InfiniteOpt, Ipopt, Plots, NLSolve, LinearAlgebra
7 end
```

Activating project at `~/julia/pluto\_notebooks`

```
1 ### Cell 2: Parameters (Sun-Earth, 3D but uz = 0)
2 begin
3     μ = 3.00348961491547e-6           # Sun = primary, Earth+Moon = secondary
4     const u_max = 0.1
5     const N_supports = 501
6     tf_lower = 50.0
7     tf_upper = 400.0
8     tf_start = 200.0
9
10    println("μ = ", μ, ", u_max = ", u_max, ", supports = ", N_supports)
11 end
```

μ = 3.00348961491547e-6, u_max = 0.1, supports = 501

0.9900265839144706

```
1 ### Cell 3: Compute L1
2 begin
3   function dOmega_dx(x, μ)
4     r1 = abs(x + μ)
5     r2 = abs(x - (1-μ))
6     return x - (1-μ)*(x + μ)/r1^3 - μ*(x - (1-μ))/r2^3
7   end
8   res = nlsolve(x -> dOmega_dx(x[1], μ), [0.99])
9   if !NLSolve.converged(res)
10     @warn "L1 solver failed"
11   end
12   x_L1 = res.zero[1]
13   println("Sun-Earth L1 at x = ", x_L1)
14   x_L1
15 end
```

Sun-Earth L1 at x = 0.9900265839144706

An InfiniteOpt Model
 Feasibility problem with:
 Finite parameters: 0
 Infinite parameter: 1
 Variables: 10
 Derivatives: 0
 Measures: 0
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.EqualTo{Float64}`: 1 constraint
 `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
 Names registered in the model: tf, ux, uy, uz, vx, vy, vz, x, y, z, τ , $\partial\Omega_x$, $\partial\Omega_y$, $\partial\Omega_z$
 Transformation backend information:
 Backend type: TranscriptionBackend
 Solver: Ipopt
 Transformation built and up-to-date: false

```

1 ### Cell 4: Model + variables (full 3D states, only ux/uy control)
2 begin
3   model = InfiniteModel(Ipopt.Optimizer)
4   set_optimizer_attribute(model, "max_iter", 5000) # Increase to 10k+
5   @infinite_parameter(model,  $\tau \in [0, 1]$ , num_supports = N_supports)
6   @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)
7
8   # Full 3D states
9   @variable(model, x, Infinite( $\tau$ ), start =  $\tau \rightarrow (1-\mu + 0.001)*(1-\tau) + x_{L1}*\tau$ )
10  @variable(model, y, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
11  @variable(model, z, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
12  @variable(model, vx, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
13  @variable(model, vy, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
14  @variable(model, vz, Infinite( $\tau$ ), start =  $\tau \rightarrow 0.0$ )
15
16  # ONLY in-plane control: ux and uy allowed, uz = 0 fixed
17  @variable(model, -u_max <= ux <= u_max, Infinite( $\tau$ ))
18  @variable(model, -u_max <= uy <= u_max, Infinite( $\tau$ ))
19  @variable(model, uz, Infinite( $\tau$ ), start = 0.0) # exists only for completeness
20  fix.(uz, 0.0; force = true) # HARD FIX: no out-of-plane thrust
21
22  # Potential gradients
23  r1(x_,y_,z_) = sqrt((x_ +  $\mu$ )^2 + y_^2 + z_^2)
24  r2(x_,y_,z_) = sqrt((x_ - (1- $\mu$ ))^2 + y_^2 + z_^2)
25   $\partial\Omega_x(x_,y_,z_) = x_ - (1-\mu)*(x_ + \mu)/r1(x_,y_,z_)^3 - \mu*(x_ - (1-\mu))/r2(x_,y_,z_)^3$ 
26   $\partial\Omega_y(x_,y_,z_) = y_ - (1-\mu)*y_/r1(x_,y_,z_)^3 - \mu*y_/r2(x_,y_,z_)^3$ 
27   $\partial\Omega_z(x_,y_,z_) = -(1-\mu)*z_/r1(x_,y_,z_)^3 - \mu*z_/r2(x_,y_,z_)^3$ 

```

28

29 `model[:∂Ωx] = ∂Ωx; model[:∂Ωy] = ∂Ωy; model[:∂Ωz] = ∂Ωz`30 `model`31 `end`

$$\int_{\tau \in [0,1]} \text{abs}(ux(\tau)) + \text{abs}(uy(\tau)) d\tau$$

```

1  ### Cell 5: Dynamics + BCs + Objective (uz = 0 enforced)
2  begin
3    # Full 3D dynamics
4    @constraint(model, ∂(x, τ) == tf * vx)
5    @constraint(model, ∂(y, τ) == tf * vy)
6    @constraint(model, ∂(z, τ) == tf * vz)
7
8    @constraint(model, ∂(vx, τ) == tf * (2*vy + model[:∂Qx](x,y,z) + ux))
9    @constraint(model, ∂(vy, τ) == tf * (-2*vx + model[:∂Qy](x,y,z) + uy))
10   @constraint(model, ∂(vz, τ) == tf * (model[:∂Qz](x,y,z) + uz)) # uz = 0 → pure
    gravity in z
11
12   # Initial: parked near Earth, zero velocity, in xy-plane
13   @constraint(model, x(0) == 1 - μ + 0.001)
14   @constraint(model, y(0) == 0.0)
15   @constraint(model, z(0) == 0.0)
16   @constraint(model, vx(0) == 0.0)
17   @constraint(model, vy(0) == 0.0)
18   @constraint(model, vz(0) == 0.0)
19
20   # Final: halt at L1, zero velocity, stay in plane
21   @constraint(model, x(1) == x_L1)
22   @constraint(model, y(1) == 0.0)
23   @constraint(model, z(1) == 0.0)
24   @constraint(model, vx(1) == 0.0)
25   @constraint(model, vy(1) == 0.0)
26   @constraint(model, vz(1) == 0.0)
27
28   # Minimum L1-fuel using only in-plane thrust
29   @objective(model, Min, ∫(abs(ux) + abs(uy), τ))
30 end

```

```
1 ### Cell 6: Solve
2 begin
3   set_silent(model)
4   println("Starting optimization... (this will take ~60-90 seconds)")
5   @time optimize!(model)
6   println("Done! Termination status: ", termination_status(model))
7 end
```

```
Starting optimization... (this will take ~60-90 seconds)
```

```
*****
This program contains Ipopt, a library for large-scale nonlinear optimization.
Ipopt is released as open source code under the Eclipse Public License (EPL).
For more information visit https://github.com/coin-or/Ipopt
*****
```

```
116.900415 seconds (15.42 M allocations: 774.844 MiB, 0.07% gc time, 4.34% compilation time: 46% of which was recompilation)
Done! Termination status: ITERATION_LIMIT
```

```
([1.001, 1.00784, 1.00854, 1.00927, 1.00861, 1.00692, 1.00368, 0.999916, 0.996966, more ,0.9
```

```
1 ### Cell 7: Extract solution
2 begin
3   opt_x, opt_y, opt_z = value(x), value(y), value(z)
4   opt_vx, opt_vy, opt_vz = value(vx), value(vy), value(vz)
5   opt_ux, opt_uy = value(ux), value(uy)
6   opt_uz = value(uz) # should be exactly zero everywhere
7   opt_tf = value(tf)
8   tau_grid = value.(supports(tau))
9   t_grid = tau_grid .* opt_tf
10
11  println("Transfer time: ", round(opt_tf, digits=2), " normalized units")
12  println("           ≈ ", round(opt_tf * 58.1328, digits=1), " days")
13  println("           ≈ ", round(opt_tf * 58.1328 / 365.25, digits=2), " years")
14  println("Max |z|: ", maximum(abs, opt_z), " (should be < 1e-8)")
15  println("Max |uz|: ", maximum(abs, opt_uz))
16
17  solution = (opt_x, opt_y, opt_z, opt_vx, opt_vy, opt_vz,
18             opt_ux, opt_uy, opt_uz, t_grid, opt_tf)
19 end
```

```
Transfer time: 207.5 normalized units
           ≈ 12062.6 days
           ≈ 33.03 years
Max |z|: 0.0 (should be < 1e-8)
Max |uz|: 0.0
```

```

1  ### Cell 8: Plots (CORRECTED)
2  begin
3      p_traj = plot(opt_x, opt_y, opt_z, lw=2.5, label="Trajectory (uz ≡ 0)",
4                  title="Earth → Sun-Earth L1 with ONLY in-plane thrust",
5                  color=:crimson)
6
7      scatter!(p_traj, [0.0], [0.0], [0.0],
8              marker=:circle, markersize=10, label="Sun")
9
10     scatter!(p_traj, [1-μ], [0.0], [0.0],
11            marker=:star5, markersize=8, label="Earth")
12
13     scatter!(p_traj, [x_L1], [0.0], [0.0],
14            marker=:diamond, markersize=9, label="L1")
15
16     # 2D plots
17     p_xy = plot(opt_x, opt_y,
18               aspect_ratio=:equal, label=false,
19               title="xy-plane projection")
20
21     p_xz = plot(opt_x, opt_z,
22               label=false,
23               title="xz-plane (should be flat line)")
24
25     # Control inputs
26     p_ctrl = plot(t_grid, [opt_ux opt_uy opt_uz],
27                  label=["ux" "uy" "uz=0"],
28                  lw=2,
29                  title="Control inputs")
30
31     display(p_traj)
32     display(plot(p_xy, p_xz, layout=(1,2), size=(900,400)))
33     display(p_ctrl)
34 end
35

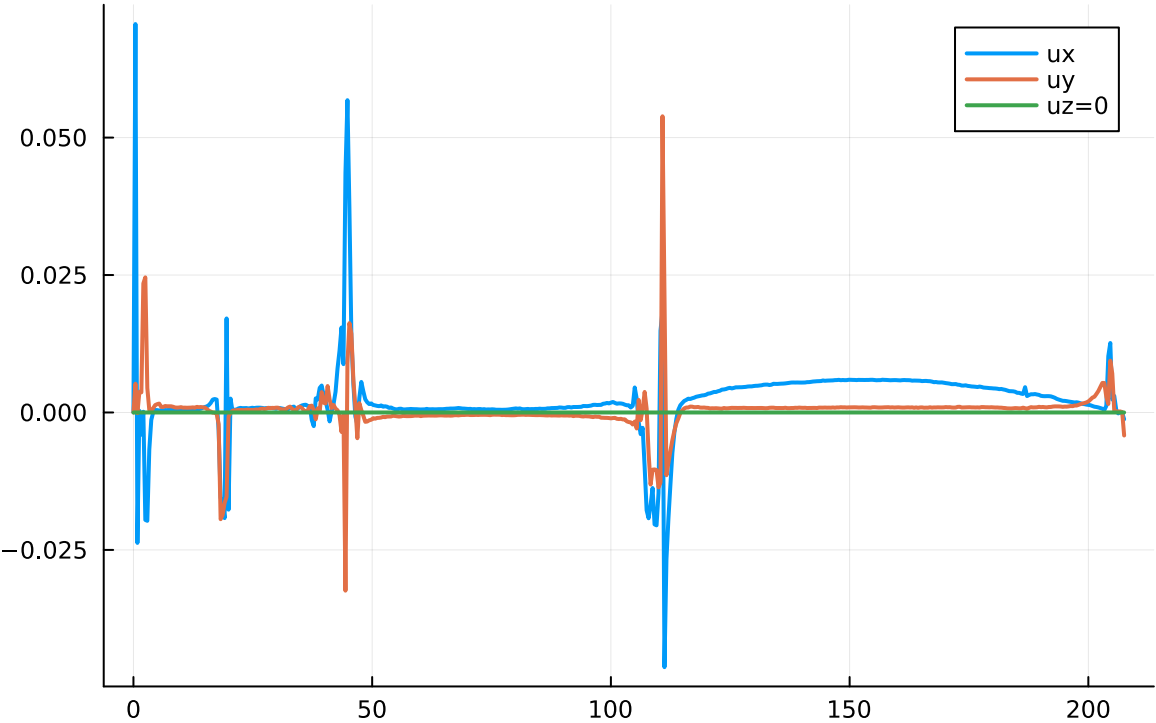
```

```

Plot{Plots.GRBackend() n=4}
Plot{Plots.GRBackend() n=2}
Plot{Plots.GRBackend() n=3}

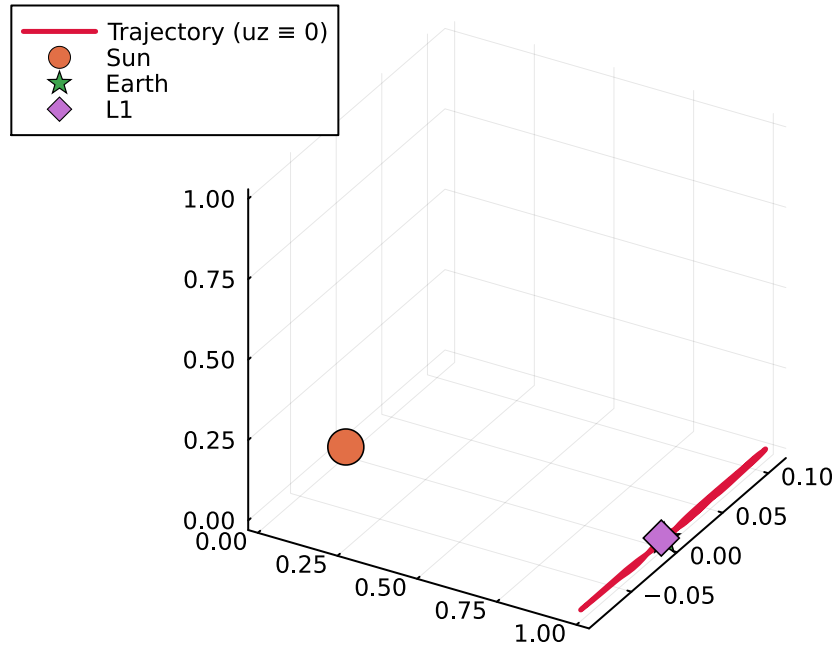
```

Control inputs



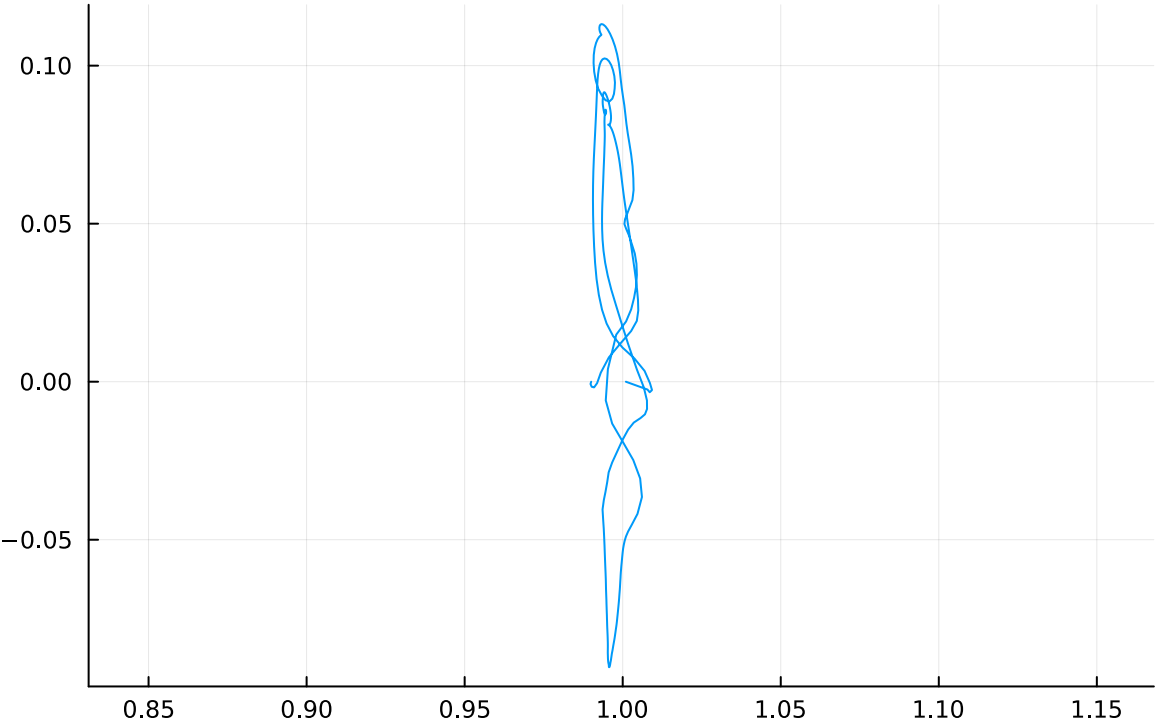
1 `p_ctrl`

Earth → Sun-Earth L1 with ONLY in-plane thrust



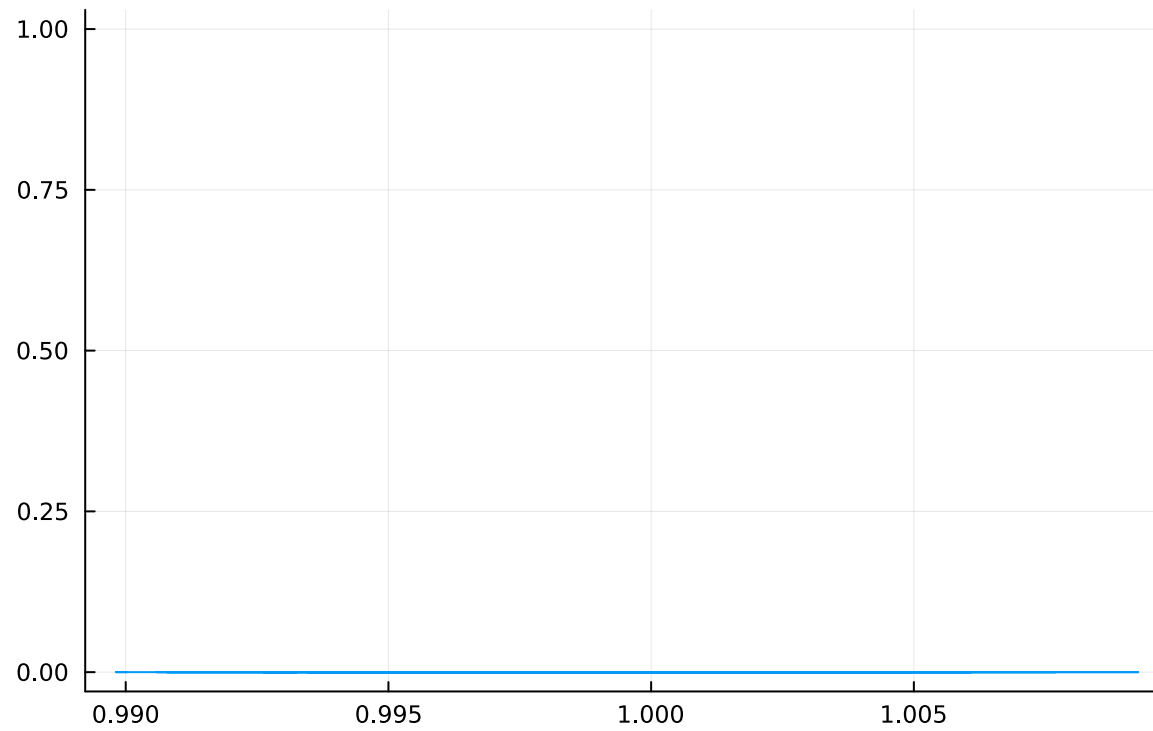
1 [p_traj](#)

xy-plane projection



1 p_xy

xz-plane (should be flat line)



1 p_xz